STEVE: Shallow Tokenized Voxel Extraction from Minecraft Images

Zhaoyuan Wan

Abstract

This project studies a constrained but practical form of image-to-structure reconstruction in Minecraft: given one rendered image of a local building region, reconstruct the visible shallow block grid and assign each occupied cell a discrete block identity. I built STEVE, a three-stage pipeline consisting of a Minecraft block visual tokenizer, depth and camera estimation modules, and a 3D voxel reconstruction network. The tokenizer converts 29,870 raw block states into 21,011 visual groups and learns a 512-entry VQ-VAE codebook with 425 active visual tokens. The reconstruction stage backprojects image observations into a voxel feature grid, rotates features into camera space for orientation-equivariant prediction, and predicts occupancy, visual tokens, material-category pairs, camera-relative yaw, visual group, geometry factor, and common block state. On a 2,248-frame validation split with no train/validation feature-key overlap, the learned model improves shallow occupancy IoU from 0.380 for the observed-surface baseline to 0.557 using ground-truth camera geometry. With a predicted-camera rotation-equivariant bridge, the model reaches 0.581 IoU and 0.735 F1 at threshold 0.36. Raw block-state recovery with closed-set common-state direct decoding and geometry-factor reranking reaches 44.1% strict raw top-1 and 61.1% category top-1 on full validation; a factorized-token variant reaches 44.0% raw top-1 with 63.3% category top-1. These results show that single-image shallow Minecraft reconstruction is feasible when visual tokenization, geometry, and voxel learning are separated, while also identifying token-level class ambiguity as the main remaining bottleneck.

1 Introduction

Minecraft worlds are made from a discrete grid of blocks, but a normal screenshot only exposes a projected, partially occluded view of that grid. The goal of this class project is to reverse part of that projection. Given one image of a local region of interest (ROI), STEVE reconstructs a shallow block-level voxel grid and exports the prediction as a Sponge schematic that can be imported back into the game.

The task is different from unrestricted 3D scene completion. I do not try to hallucinate an entire building, hidden interiors, or the far side of an object that is never visible. The target is the shallow visible structure: surface cells and a small number of cells behind visible surfaces that can plausibly be inferred from a single view. This is still a challenging machine learning problem because Minecraft block states are long-tailed, many raw states are visually identical, block orientation interacts with camera orientation, and depth/camera errors can move an otherwise correct block into the wrong voxel.

The project therefore separates the problem into three parts. First, a block tokenizer learns discrete visual tokens from canonical multi-view renders of Minecraft block states. Second, image geometry modules estimate metric depth and camera pose, while controlled experiments can also use ground-truth geometry. Third, a 3D convolutional reconstruction model consumes backprojected voxel features, optionally rotates them into camera space for orientation-equivariant prediction, and predicts occupancy plus token/state labels through multiple heads. This design makes the learning target more

realistic than direct raw-state classification over thousands of classes and provides explicit places to evaluate geometry, token quality, and voxel reasoning.

The main contributions of the project are:

- a data pipeline using the project assets, including `block-exporter` outputs for block vocabulary construction and `building-capture` outputs for ROI/frame supervision;
- a learned block visual vocabulary that maps 29,870 raw states to 21,011 rendered visual groups and then to 425 active VQ tokens, with rotation-canonical grouping and camera-relative yaw targets;
- a voxel reconstruction model with occupancy, token-material-pair, yaw, material, visual-group, geometry-factor, and common-state heads, trained with closed-set target masking, token-confusion margin, and residual 3D context;
- a predicted-camera rotation-equivariant bridge that averages four 90-degree yaw gauges for robust occupancy prediction;
- a full validation protocol over 2,248 frames with explicit geometry baselines, split-audit checks, predicted-camera experiments, and raw-state diagnostics including recoverable-miss analysis and per-error-source decomposition;
- a standalone raw-image inference pipeline that auto-generates depth, camera, and ROI metadata from a single image and user mask, plus an honest benchmark suite with per-sample diagnostic metrics (visible-surface voxel agreement, overfill ratios, bounding-box comparisons, and material-collapse detection).

2 Related Work

Single-view 3D reconstruction. Early neural single-view reconstruction systems such as 3D-R2N2 predict voxel grids from images using recurrent or convolutional encoders (1). Later methods improve output resolution through octree or sparse representations (6). STEVE uses a simpler dense local voxel grid because the ROI is bounded and Minecraft's native representation is already discrete. Unlike object reconstruction benchmarks, the output here is not only geometry but also a block identity for every occupied voxel.

Depth and camera estimation. Single-image depth prediction is a standard supervised learning task (2). In this project, depth is not the final output: it is an intermediate signal used to backproject pixels into the ROI grid. I trained a lightweight U-Net-style depth predictor (5) and a compact camera regressor. For controlled evaluation, I also report ground-truth-depth and ground-truth-camera settings to separate reconstruction quality from upstream geometry errors.

Discrete visual representations. Vector-quantized autoencoders learn discrete latent codes for high-dimensional observations (7). STEVE applies this idea to rendered Minecraft block states. The tokenizer does not attempt to recover every raw game state directly. Instead, it learns visual tokens from RGB, mask, depth, and normal renderings, then later decoding stages map token predictions back toward raw states using rotation-canonical grouping, yaw, material, visual-group, and common-state heads. Recent work on video tokenization (8) and image quantization (3) has shown that well-designed tokenizers are critical for reconstruction quality; STEVE's block tokenizer similarly trades off codebook size against reconstruction fidelity, finding that 425 of 512 codes are active with a final perplexity of approximately 155.

Voxel games and neural scene representations. Neural radiance fields (4) represent scenes continuously and excel at view synthesis, but Minecraft already supplies a discrete, editable representation. The output needed for this project is not a rendered novel view but an importable block schematic. This makes voxel occupancy, state prediction, and schematic export more relevant than continuous radiance estimation.

3 Problem Definition

Each sample contains a local ROI with dimensions up to a small fixed block grid, a rendered RGB frame, an ROI mask, metric depth, camera metadata, and shallow voxel targets. Let I be the input image and let $V \in \{0,1\}^{X \times Y \times Z}$ denote shallow occupancy in ROI coordinates. For every occupied cell c, the data also provides a block target, represented at multiple granularities: raw block state, visual group, VQ token, material category, camera-relative yaw class when orientation is meaningful, and a common-state class for frequent raw states.

The model predicts

$$\hat{V}, \hat{t}, \hat{y}, \hat{m}, \hat{g}, \hat{s}$$

where $\hat{t}$ is a token-material-pair class (decoded into canonical token logits plus material logits), $\hat{y}$ is camera-relative yaw, $\hat{m}$ is a material category, $\hat{g}$ is visual group, and $\hat{s}$ is a common raw state class. A geometry-factor head is trained as an auxiliary target and used only at export time for reranking, not included in the main prediction vector. The most important metrics are occupancy IoU/F1, token accuracy on occupied cells, joint occupied-token accuracy, and strict raw-state top-1/top-5 when a raw-state decoder is used.

This framing intentionally avoids impossible labels. Many raw states encode invisible properties such as waterlogged flags, redstone power, or note-block settings. The paper therefore reports raw-state accuracy as a diagnostic decoding metric, not as the only measure of reconstruction quality.

4 Method

4.1 Overview

Figure 1 summarizes the implemented pipeline. Minecraft resources first produce a canonical block vocabulary. A VQ-VAE learns visual block tokens from multi-view renders. Separately, captured building frames provide RGB images, masks, depth, camera pose, and voxel targets. During reconstruction, pixels are backprojected into the ROI grid, producing a dense voxel feature tensor. The feature grid is optionally rotated into camera space so that orientation-dependent predictions become camera-relative. A 3D CNN with residual blocks, optional pooled context attention, and multiple prediction heads then predicts shallow occupancy and block-related labels. A raw-state decoder combines token, material, yaw, visual-group, geometry-factor, and common-state predictions with frequency priors to produce an importable `.schem` file.

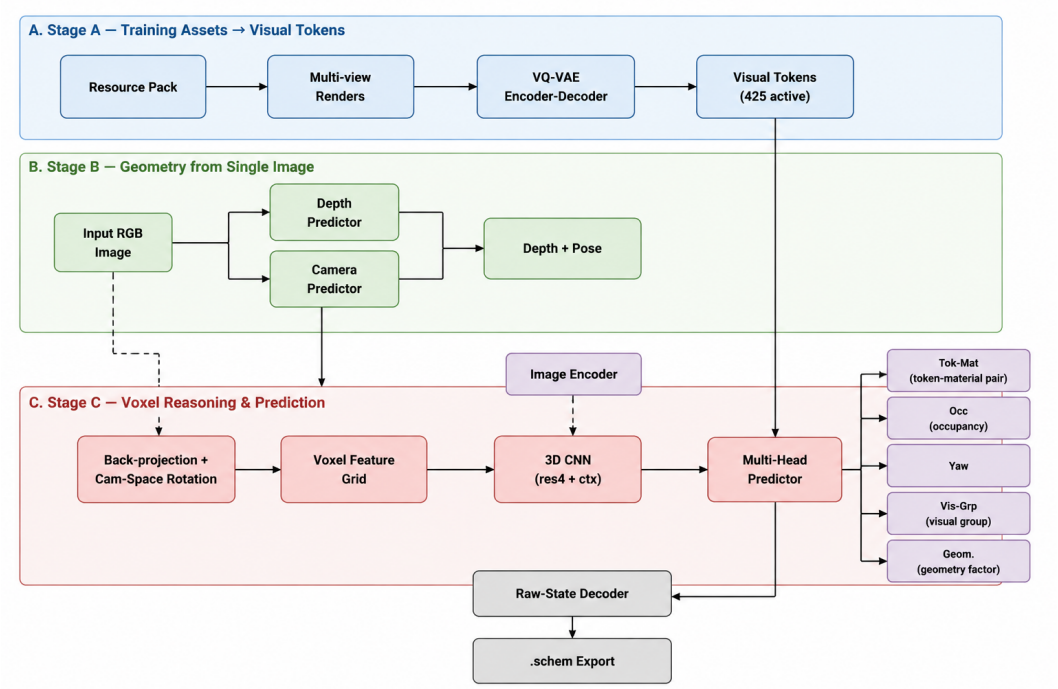


Figure 1: STEVE pipeline. Stage A trains a VQ-VAE block tokenizer from Minecraft resource-pack renders (top, blue). Stage B estimates metric depth and camera pose from a single image (middle, green). Stage C backprojects image evidence into a voxel grid, rotates into camera space, and predicts occupancy plus block-identity heads (bottom, red). A raw-state decoder combines multi-head predictions with frequency priors to produce an importable Minecraft schematic. The dashed arrow indicates optional image-context injection.

4.2 Block Visual Tokenizer

The tokenizer is trained on `block-exporter` assets. The exporter uses Minecraft's own model and resource loading rather than a hand-written model parser. For each state, it writes seven canonical views: front, back, left, right, top, isometric front-left, and isometric front-right. Each view contains RGB, alpha/mask, depth, and normal supervision. The preprocessing pass reads 29,870 state directories, filters 32 unsupported dynamic states, and merges exact rendered duplicates into 21,011 visual groups.

The tokenizer is a multi-view VQ-VAE (7). For a block state, each view is encoded by shared 2D convolutions, view features are averaged, and a 128-dimensional latent vector is quantized by an EMA codebook. The decoder reconstructs all views with RGB, mask, depth, and normal channels. The training loss is

$$L_{\text{tok}} = L_{\text{rgb}} + L_{\text{mask}} + 0.5L_{\text{depth}} + 0.25L_{\text{normal}} + 0.25L_{\text{vq}}.$$

The selected tokenizer uses a 512-entry codebook. Evaluation found 425 active codes and 87 dead codes; the final logged perplexity was approximately 155.0. Figure 2 shows example tokenizer reconstructions.

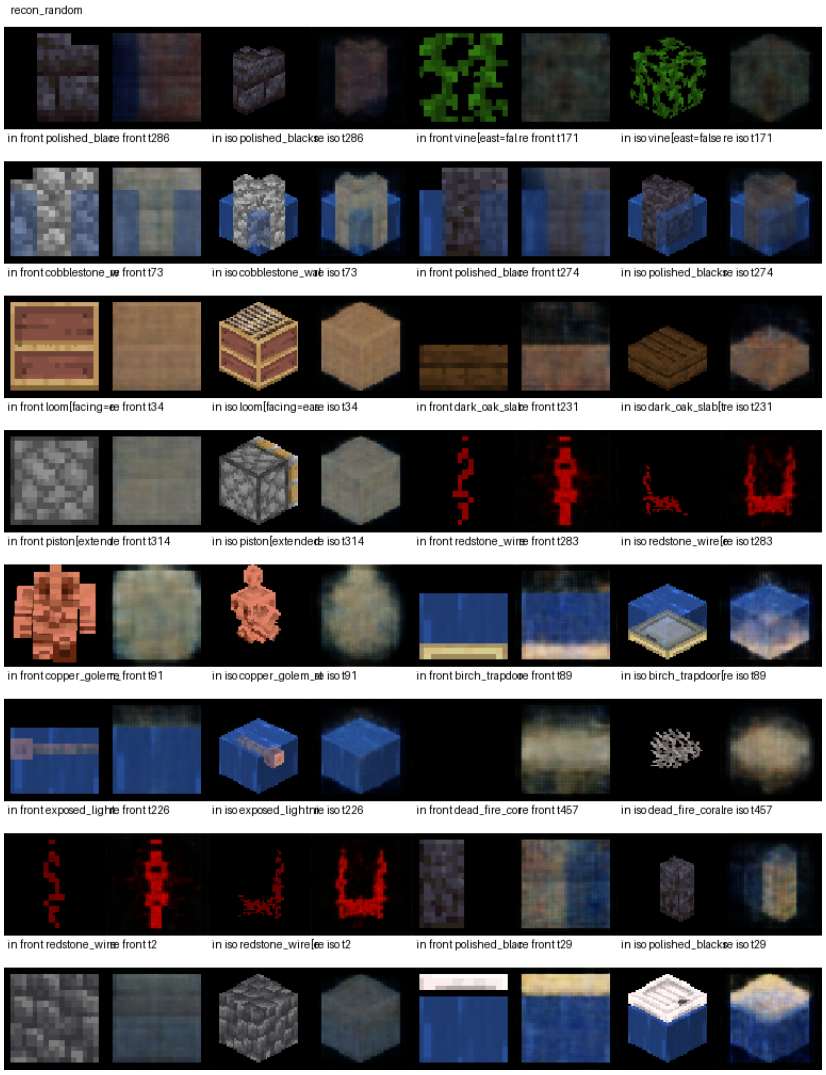


Figure 2: Cropped random tokenizer reconstruction sheet from the selected VQ-VAE run. The tokenizer learns compact visual codes from canonical Minecraft block renders before voxel reconstruction is trained.

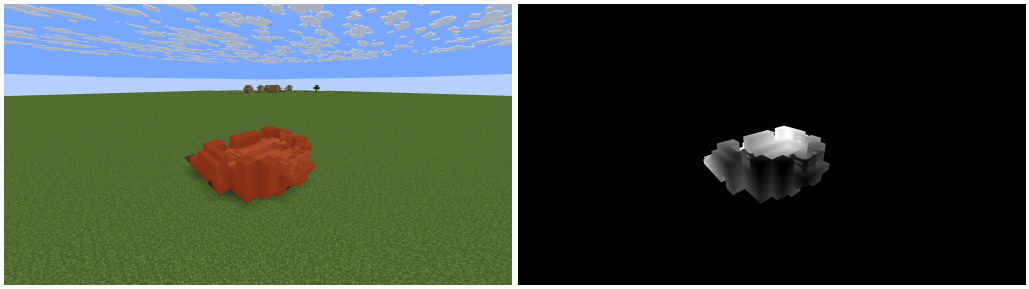
4.3 Rotation-Canonical Targets

Minecraft states include many orientation properties, including `facing`, `axis`, `rotation`, and wall/stair connection states. Directly predicting a raw state would force the network to treat the same visual object under different yaw rotations as unrelated classes. STEVE therefore separates canonical visual identity from orientation. The target builder maps raw states to canonical token/material representations and writes a camera-relative yaw target where orientation is meaningful. At export time, camera-relative yaw is converted back to absolute yaw using the predicted or known camera yaw bin.

4.4 Backprojection and Voxel Features

For every frame, the reconstruction pipeline backprojects mask-supported pixels into the ROI grid using metric depth and camera pose. The resulting `voxel_features.npz` tensor has layout $[X, Y, Z, C]$. The implemented channels include hit count, mean RGB, mean depth, depth variance, mean view direction, mask support, ray-candidate features behind visible hits, and global masked image statistics. The ray-candidate channels help distinguish direct surface observations

from shallow continuation cells along the camera ray. Figure 3 shows one debug example of image mask and depth processing.



(a) RGB with ROI mask (b) Metric depth preview

Figure 3: Backprojection inputs used to build voxel observations. Depth and camera pose convert image evidence into ROI-grid features.

4.5 Depth and Camera Modules

The depth predictor is a lightweight U-Net. A four-layer convolutional encoder downsamples the RGB image, and a transposed-convolution decoder upsamples with skip connections. It is trained with masked L1 loss on valid ROI pixels. On the validation split, the best depth model reaches 7.11 block MAE, 13.70 block RMSE, and $\delta_1 = 0.320$.

The camera predictor uses a compact image encoder followed by an MLP. The target contains yaw sine/cosine, relative camera-position sine/cosine, pitch, and distance. Because many building captures are symmetric under 90-degree rotations, the loss can be evaluated under four yaw rotations and take the minimum. The best validation checkpoint has rotation-equivalent yaw MAE of 17.66 degrees, pitch MAE of 13.10 degrees, and distance MAE of 4.94 blocks.

4.6 Voxel Reconstruction Network

The main Stage B model is a 3D CNN over the backprojected feature grid. The final architecture uses three convolutional body blocks, four 3D residual blocks, an optional pooled voxel self-attention context layer, and multiple prediction heads. A small image encoder injects global image context into every voxel, and an auxiliary camera head regresses camera parameters from the same image embedding. Before the CNN forward pass, the entire feature grid and all targets can be rotated into camera space so that orientation-dependent predictions become camera-relative; after prediction, outputs are rotated back to ROI/world coordinates for export.

The output heads are:

- occupancy logits for shallow structure;
- token-material-pair logits, which decode into canonical token logits plus material logits (the pair target replaces a plain 512-way token target to reduce cross-material ambiguity);
- yaw logits for camera-relative orientation;
- material-category logits (auxiliary);
- visual-group logits (auxiliary, sparse vocabulary);
- geometry-factor logits (auxiliary, sparse vocabulary);
- common-state logits for frequent raw states (auxiliary, sparse vocabulary, confidence-gated direct decoding at export time).

Training uses binary cross entropy for occupancy and cross entropy for valid occupied-cell categorical targets. Empty cells use `ignore_index=-100` for token/state losses. Closed-set target masking removes tail raw states from token/material/yaw loss when their training-set frequency is below a threshold. A token-confusion margin loss penalizes high-frequency confused token pairs. The raw-state decoder combines predicted common state, token/material candidates, visual-group and

geometry-factor reranking, state priors, and yaw representative mappings. This decoder is intentionally reported separately from occupancy and token metrics because exact raw-state recovery is partly limited by label ambiguity in the VQ tokenizer.

4.7 Raw Image Inference with Auto-Generated Metadata

While the main reconstruction pipeline depends on capture metadata (metric depth, camera pose, ROI selection box) from the `building-capture` tool, STEVE also supports standalone raw-image inference for classwork-style usage. In this mode, the system receives only an RGB image and an optional user-drawn mask. All intermediate metadata—depth, camera parameters, and the ROI selection box—are predicted or estimated internally, then written as standard metadata files that the existing backprojection and export code can consume without modification.

The auto-estimation pipeline works as follows. Depth is predicted from the RGB image using the same U-Net depth predictor, calibrated so the masked median depth aligns with the predicted camera distance. Camera yaw, pitch, and distance are estimated via the camera regressor. The ROI bounding box is computed from the mask bounding box plus projected depth. All estimated metadata is written as structured JSON and binary files (`selection_metadata.json`, `pose.json`, `metric_depth.bin`) before backprojection, so the reconstruction path is identical to the capture-based pipeline. A browser-based UI allows users to draw the ROI mask interactively, and a command-line wrapper is also available.

To evaluate raw-image quality honestly, a benchmark script produces per-sample diagnostic reports beyond simple occupancy counts and top-view footprint IoU. The benchmark collects:

- visible-surface voxel IoU, precision, and recall against backprojected surface evidence;
- overfill ratios (predicted occupied cells divided by surface evidence cells);
- bounding-box size comparisons between prediction and surface evidence (x, y, z dimensions and column counts);
- material-collapse indicators: dominant-state ratio, mossy-block ratio, concrete ratio, and light-gray-concrete ratio.

These diagnostics intentionally expose common failure modes in the raw-image path: ROI size overestimation or underestimation from mask bounding box plus imprecise depth, material prediction collapsing toward frequent ambiguous states (mossy blocks, light-gray concrete), and background pixel contamination from over-broad masks. The benchmark also supports target-enabled evaluation when a known schematic is available, though target occupancy IoU is low by design because the raw-image path only reconstructs visible evidence, not hidden interiors or backsides. The current default checkpoint, fine-tuned on calibrated-auto single-image assumption pairs, reached reliable footprint coverage on 48 family-balanced standalone images with no unsupported columns and mean top-footprint IoU of 0.985.

5 Experiments

5.1 Data and Splits

The project uses two asset sources. The block vocabulary comes from `assets/block_vocab.zip` and its extracted Minecraft 26.1.2 render data. Building reconstruction data comes from `assets/building-capture`. The validation protocol used for the main paper pass has 8,614 training frames and 2,248 validation frames. A protocol audit found zero shared train/validation feature keys, and the full-validation manifest matched all 2,248 selected validation features.

The target grid stores shallow occupancy, token target, visual-group target, yaw target, material target, and raw-state information when available. The raw-state prior contains 29,838 kept states, 25,702 orientable states, and 373 common states with frequency at least 100. The top 100 common states cover 85.4% of prior cells; the top 500 cover 97.9%.

5.2 Metrics

For occupancy, I report precision, recall, F1, and IoU over shallow target cells:

$$\text{IoU} = \frac{TP}{TP + FP + FN}, \qquad F1 = \frac{2TP}{2TP + FP + FN}.$$

For identity, I report token accuracy on valid occupied cells, joint occupied-token accuracy on cells that are both occupied and token-correct, and raw-state strict top-1/top-5 after schematic-style decoding. Category top-1 is included because many block states differ only by specific material or state properties.

5.3 Geometry and Occupancy Results

Table 1 gives the main geometry results. The observed-surface baseline simply marks backprojected surface hits as occupied. It has high precision but low recall because shallow cells behind the visible surface are missing. A one-cell dilation improves recall but creates too many false positives. The learned voxel model improves both baselines substantially.

Table 1: Full-validation occupancy and joint occupied-token results over 2,248 frames. Surface baselines are occupancy-only and do not predict tokens. "Rot4" denotes the predicted-camera rotation-equivariant bridge. The rot4 row matches GT-camera best to four significant digits.

Setting	Thr.	IoU	F1	Prec.	Recall	Joint tok.
Observed surface	0.10	0.380	0.551	0.853	0.407	–
Surface + dilation	0.10	0.333	0.499	0.390	0.694	–
GT depth + GT camera	0.30	0.557	0.715	0.594	0.898	0.538
GT depth + GT camera (best)	0.36	0.581	0.735	0.637	0.869	0.539
GT depth + predicted cam. (rot4)	0.36	0.581	0.735	0.637	0.869	0.539

The GT-depth and GT-camera learned model reaches 0.557 IoU and 0.715 F1 at threshold 0.30. At threshold 0.36, IoU reaches 0.581. The predicted-camera model with the rotation-equivariant bridge reaches the same IoU of 0.581 and F1 of 0.735 at threshold 0.36, confirming that moderate camera-yaw error does not destroy voxel reconstruction when the prediction uses per-gauge averaging and camera-relative targets. A higher-threshold profile at 0.45 can reach IoU 0.611 but at the cost of reconstruction accuracy (joint token drops to 0.525).

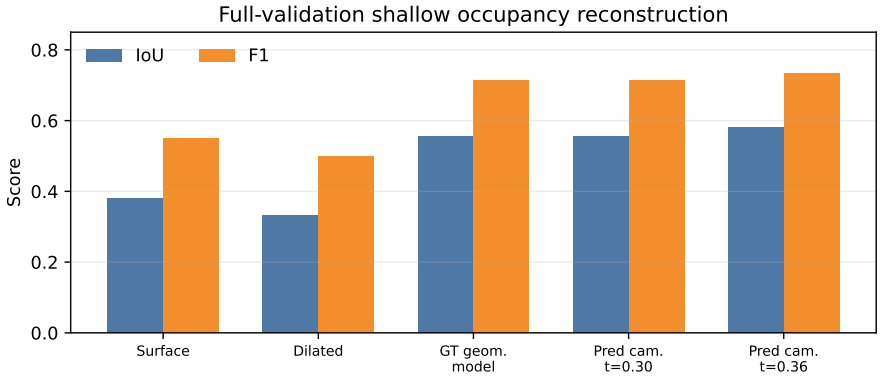


Figure 4: The learned voxel model improves over geometry-only baselines. Surface hits are precise but incomplete; dilation improves recall but lowers precision.

5.4 Threshold Tradeoff

Figure 5 plots the predicted-camera threshold sweep. Increasing the threshold raises precision and IoU/F1 up to the evaluated maximum of about 0.50, but it lowers reconstruction accuracy measured

over the full shallow target because some true cells are removed. I therefore report both a balanced threshold, 0.36, and the same-threshold comparison at 0.30. The same-threshold comparison is the cleaner ablation against GT camera; the tuned threshold is the better deployment setting when the goal is geometry quality.

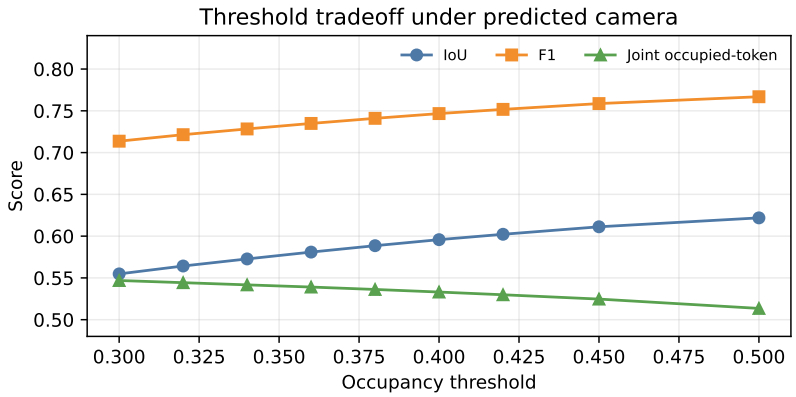


Figure 5: Predicted-camera threshold sweep. IoU/F1 increase with stricter occupancy thresholds, while full reconstruction accuracy decreases because recall drops.

5.5 Token and Raw-State Results

Token prediction is easier than exact raw-state recovery. In the GT-camera full-validation setting, token accuracy is 56.7%, joint occupied-token accuracy is 59.8%, and yaw accuracy is 64.1% on valid yaw cells. The final model with the window-context attention, common-state auxiliary head, geometry-factor head, and material/visual-group reranking improves these to 59.9% token accuracy and 70.6% yaw accuracy.

Table 2 shows raw-state decoding diagnostics for the final model configuration. The key result is that common-state direct decoding—where the common-state head's prediction overrides the representative candidate when its confidence exceeds a threshold—substantially improves strict raw top-1. With direct decoding, confidence-gated visual-group reranking, and geometry-factor reranking, the best full-validation raw top-1 reaches 44.1% and category top-1 reaches 61.1%. A factorized-token variant using the same model backbone but a different token target reaches 44.0% raw top-1 with 63.3% category top-1, showing that the common-state head provides complementary disambiguation regardless of whether the token target is a plain VQ class or a token-material pair.

Error analysis shows that the dominant remaining failure is token-level prediction error (35.5% of valid cells), which is almost never recoverable through top-5 candidate reranking (top-5 recovery rate: 0.01%). In contrast, representative-selection errors are partially recoverable: 12.6% of top-1 failures recover the correct state in top-5. This confirms that future improvements should target the token/classifier itself rather than the export-side ranking pipeline.

Table 2: Raw-state decoding diagnostics on full validation (2,248 frames). All accuracies are percentages over valid shallow cells.

Setting	Frames	Raw@1	Raw@5	Cat.@1	Pred-occ Raw@1
No common direct (baseline)	2248	34.3	45.3	54.7	35.8
Unified + common direct conf 0.5	2248	43.0	49.6	61.0	45.1
Unified + geom. factor + common direct	2248	44.1	52.3	61.1	46.0
Factorized + common direct conf 0.55	2248	44.0	55.2	63.3	46.1

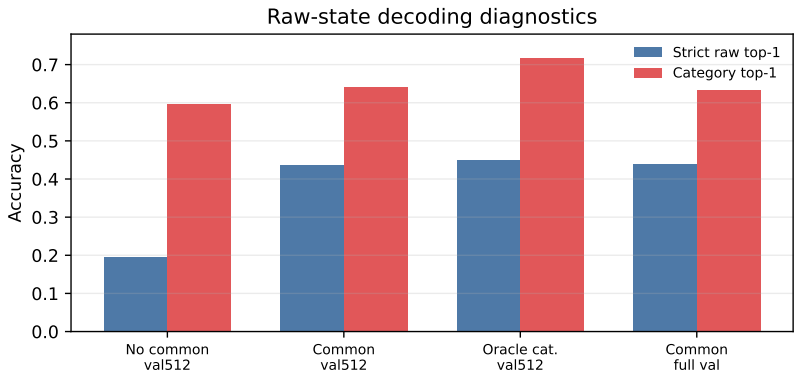


Figure 6: Common-state direct decoding and geometry-factor reranking improve exact block-state recovery, but token-level prediction remains the main bottleneck.

5.6 Qualitative Reconstruction Results

Figures 7–9 show in-game screenshots of exported reconstruction schematics. I traced each screenshot back through its companion `.schem.json` file to recover the original captured frame, ROI mask, and depth image from `assets/building-capture`. The known-camera and predicted-camera exports use the rotation-equivariant bridge with common-state direct decoding and geometry-factor reranking. The comparison shows that the predicted camera preserves the main structure silhouette and block-material families with moderate cell-density reduction.

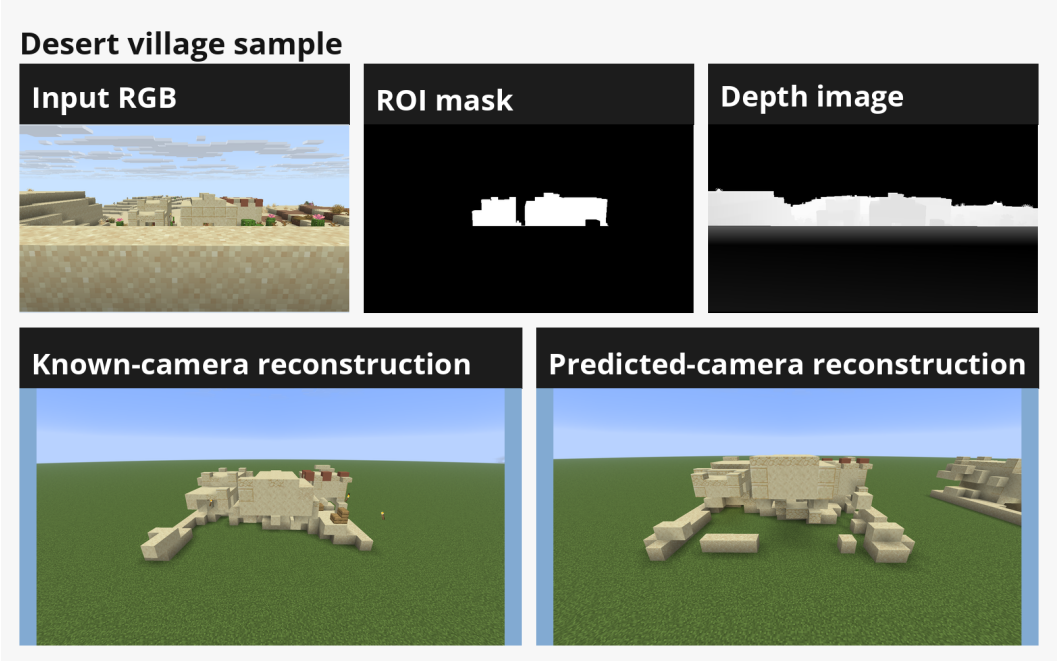


Figure 7: Desert village reconstruction. The known-camera export contains 583 occupied cells; the predicted-camera export contains 528 occupied cells. The main structure silhouette and block families are preserved under predicted camera.

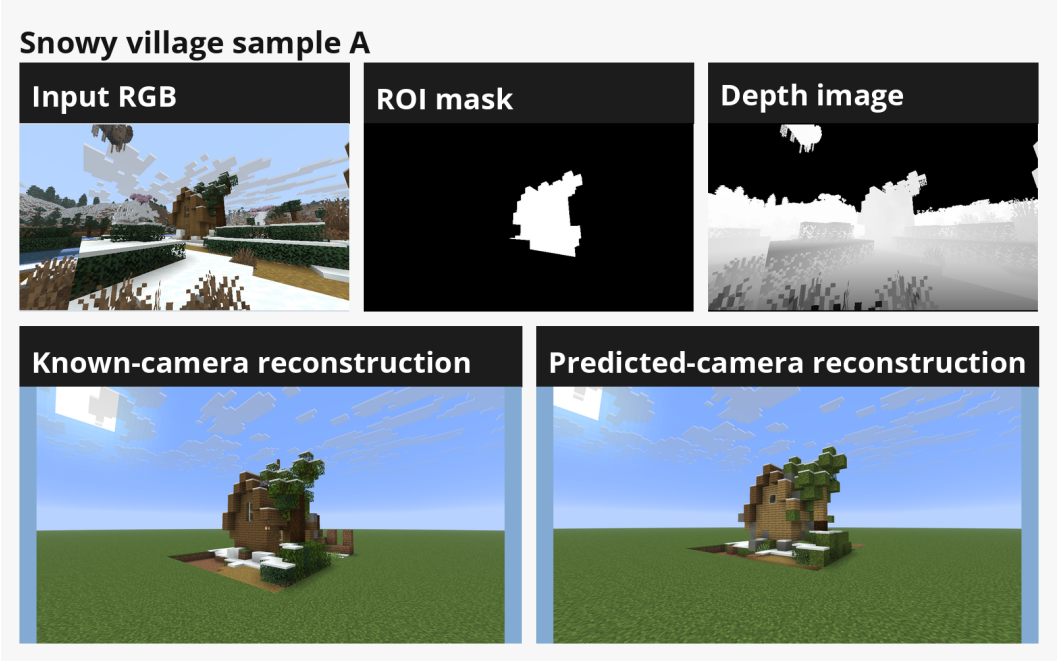


Figure 8: Snowy village reconstruction A. The known-camera export preserves the main wooden house, tree mass, snow, and ground patch with 984 occupied cells; the predicted-camera export is sparser with 690 occupied cells.

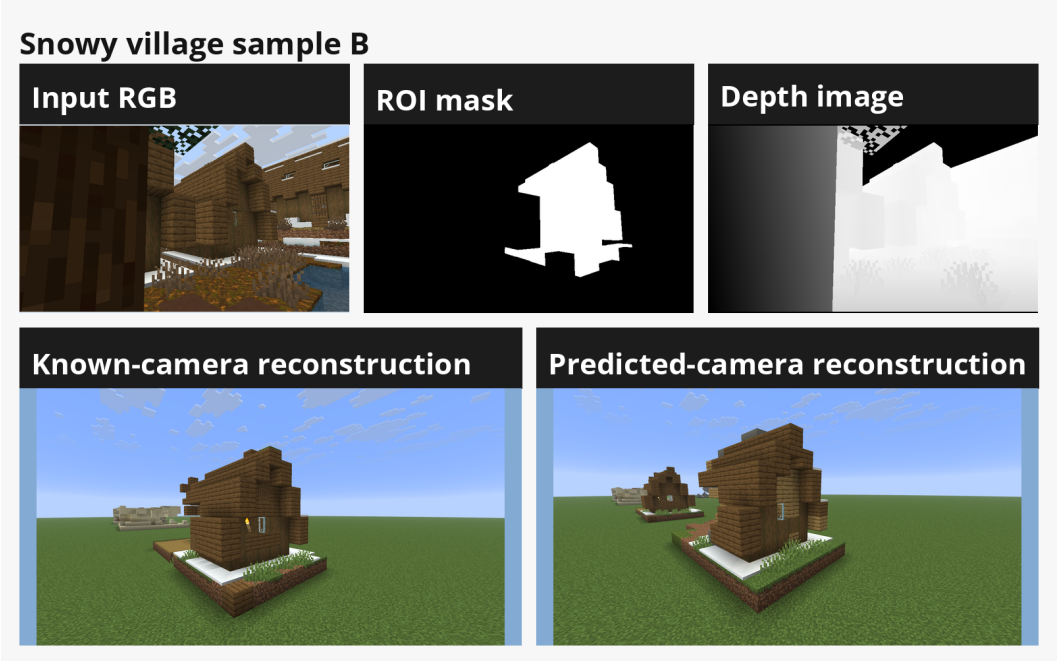


Figure 9: Snowy village reconstruction B. The known-camera export contains 378 occupied cells and the predicted-camera export contains 311 occupied cells; both recover the dominant cabin silhouette and block-material families.

6 Ablation and Analysis

6.1 Architecture Ablations

Table 3 summarizes key ablations on raw-state decode metrics. All entries use GT-depth-derived voxel observations, the rotation-canonical + camera-relative yaw target, closed-set target masking (min-count 100), and the recommended export configuration unless otherwise noted.

Table 3: Ablation study on full-validation raw-state decoding (threshold 0.10, closed-set 100). Raw@1 = strict raw top-1 over valid shallow cells; Cat.@1 = strict category top-1.

Ablation	Raw@1	Raw@5	Cat.@1	Token acc.
Plain CNN, no residual	30.8	42.5	50.0	–
+ residual blocks (res4)	35.3	46.6	55.8	59.9
+ camera-space grid	35.1	45.8	54.7	56.7
+ material head	34.3	44.5	53.9	–
+ closed-set target mask	35.3	46.6	55.8	–
+ token-material-pair	38.0	47.0	57.5	57.2
+ common-state head (direct)	41.8	49.4	59.5	–
+ window context + geom. factor	**44.1**	**52.3**	**61.1**	59.9

Adding residual 3D context is the single strongest architecture change (+4.5 raw top-1). The token-material-pair target replaces the plain 512-way VQ token with a material-conditioned class, reducing cross-material confusion (+2.7 raw top-1). Direct common-state decoding at export time provides the largest remaining single-stage improvement (+3.5 raw top-1) because it bypasses the overloaded VQ token bucket for high-confidence state predictions. Window context and geometry-factor reranking add a final +2.3 raw top-1.

6.2 Why Not Direct Raw-State Classification?

Raw-state classification is attractive because it directly matches the final schematic format. However, Minecraft raw states include many visually invisible or weakly visible properties. A visual duplicate audit found thousands of duplicate raw states after rendered grouping. The tokenizer also showed mixed tokens: some tokens cluster blocks by shape, others by material, and some mix visually similar families. Direct raw-state classification would penalize the model for failing to recover properties that do not appear in the image.

The common-state head is a practical compromise. It predicts frequent states when there is enough evidence and falls back to token/material/visual-group decoding otherwise. This is why the common-state rows in Table 2 outperform the no-common diagnostic for strict raw top-1.

6.3 Geometry Baselines

The observed-surface and dilation baselines are important protocol checks. Surface IoU of 0.380 confirms that backprojection is aligned and precise, while its recall of 0.407 shows that surface hits alone are not enough for shallow reconstruction. Dilation recall rises to 0.694, but precision falls to 0.390 and IoU drops to 0.333. The learned model learns a better precision-recall balance than either hand rule.

6.4 Predicted Camera

The camera predictor has rotation-equivalent yaw MAE of 17.7 degrees. A rotation-equivariant bridge evaluates the four possible 90-degree yaw gauges implied by each predicted camera yaw and averages probabilities/logits across gauges. This approach avoids committing to an arbitrary absolute yaw and significantly improves occupancy prediction under predicted camera. The predicted-camera balanced profile at threshold 0.36 reaches IoU 0.581, F1 0.735, matching the GT-camera best. However, exact yaw-dependent block state recovery is still weaker under predicted camera, which is visible in the

lower yaw accuracy for the absolute-yaw case. The rotation-equivariant bridge is the recommended default for predicted-camera inference.

6.5 Depth Prediction

The depth model is trained and evaluated, but the main full-validation reconstruction table uses GT-depth-derived voxel observations to isolate the voxel model and camera ablations. The depth MAE of 7.11 blocks is large relative to small structures, so a fully predicted-depth pipeline would need more training, stronger image encoders, or temporal/multiview fusion before it could match the controlled geometry results. The current project therefore treats full predicted-depth reconstruction as future work rather than overclaiming it.

7 Implementation

The implementation is organized as a Python package under `STEVE/`. The tokenizer code lives in `tokenizer/steve_tokenizer/`; the reconstruction code lives in `reconstruction/steve_reconstruction/`; command-line scripts live in `scripts/`. Important commands include `steve-tokenizer-preprocess`, `steve-tokenizer-train`, `steve-build-voxel-observations`, `steve-train-stage-b`, `steve-eval-stage-b`, and `steve-export-stage-b-schematic`. For standalone image inference, `steve-reconstruct-raw-image` reconstructs from a single RGB image with auto-generated depth, camera, and ROI metadata; `steve-benchmark-raw-images` runs batch evaluation with honest diagnostics (visible-surface voxel agreement, overfill ratios, bounding-box comparisons, and material-collapse detection); `steve-raw-image-web` launches a browser UI for interactive mask drawing and reconstruction.

The schematic export path writes Sponge schematic v2 files. The qualitative figures in this paper were rendered from exported schematics and paired with their originating source frames by reading the export JSON metadata. One larger Terralith desert outpost smoke export produced a $27 \times 15 \times 25$ schematic with 3,253 occupied cells, 78 palette entries, and 47 unique predicted token classes. This confirms that the model output can be converted back into the native Minecraft block format.

8 Limitations

The project has several limitations. First, the best reconstruction numbers are for GT-depth-derived voxel observations; the depth predictor reaches 7.11 block MAE and the fully predicted-depth pipeline is not yet strong enough to match controlled geometry results. The raw-image inference path provides auto-generated metadata as a practical bridge, but its quality depends on depth/camera accuracy and mask quality. Second, raw-state recovery remains limited by token-level class ambiguity. The VQ tokenizer assigns visually similar but materially different blocks (stone/dirt/grass, cobblestone/stone, sand/sandstone) to the same mixed token bucket. Common-state direct decoding and material/yaw/geometry reranking mitigate this at export time, but the dominant error—primary token wrong—accounts for 35.5% of valid cells and is almost never recoverable through top-5 candidate reranking. A cleaner tokenizer or factorized target representation would be needed for a larger improvement. Third, the validation set is synthetic and collected from the same Minecraft setup and resource pack; cross-resource-pack generalization was not tested. Fourth, single-image reconstruction cannot observe hidden interiors or backsides, so STEVE should be evaluated as shallow visible reconstruction rather than complete structure recovery. Fifth, the factorized token variant does not yet outperform the unified model on all metrics simultaneously; it improves raw top-1 and category accuracy but its token accuracy is near zero because its primary decode path is common-state direct override. Sixth, the raw-image benchmark diagnostics (visible-surface voxel agreement, overfill, bounding-box, and material-collapse metrics) surface common failure modes—ROI estimation errors, material collapse to mossy/concrete states, and background contamination—that are hidden behind top-view footprint IoU alone, and these honest diagnostics indicate that raw-image reconstruction quality still has substantial room for improvement.

9 Conclusion

STEVE demonstrates that a structured machine learning pipeline can reconstruct shallow Minecraft block structure from images. The most reliable result is geometry: the learned model improves full-validation occupancy IoU from 0.380 for observed-surface backprojection to 0.557 with GT camera and 0.581 with predicted camera using a rotation-equivariant bridge. The visual tokenizer and multi-head voxel model also recover useful block identity signals. With closed-set common-state direct decoding and geometry-factor reranking, strict raw top-1 reaches 44.1% and category top-1 reaches 61.1% on full validation; a factorized-token variant reaches 44.0% raw top-1 with 63.3% category top-1. Error decomposition shows that token-level prediction error (35.5% of cells) is the dominant remaining blocker and is almost never recoverable through export-side ranking, pointing to tokenizer or target representation cleanup as the next high-value direction. The main lesson is that separating visual tokenization, camera/depth geometry, and voxel learning makes the task tractable and debuggable. The next step would be stronger depth prediction and multiview fusion so that schematic export can rely less on privileged geometry, combined with a factorized or material-aware tokenizer that produces cleaner token targets.

Acknowledgements

This project was developed for CS3308. The assets came from the local Minecraft block vocabulary and building-capture pipelines, including the `block-exporter` and `building-capture` tools.

References

[1] Christopher B. Choy, Danfei Xu, JunYoung Gwak, Kevin Chen, and Silvio Savarese. 3d-r2n2: A unified approach for single and multi-view 3d object reconstruction. In *European Conference on Computer Vision*, pages 628–644. Springer, 2016.

[2] David Eigen, Christian Puhrsch, and Rob Fergus. Depth map prediction from a single image using a multi-scale deep network. In *Advances in Neural Information Processing Systems*, volume 27, 2014.

[3] Doyup Lee, Changhwan Kim, Saehwan Jeong, Donghyun Kim, Eunyi Kim, and SeungHwan Yoon. ImagenGT: Autoregressive image generation without vector quantization. In *International Conference on Machine Learning*, 2022.

[4] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1):99–106, 2021.

[5] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical Image Computing and Computer-Assisted Intervention*, pages 234–241. Springer, 2015.

[6] Maxim Tatarchenko, Alexey Dosovitskiy, and Thomas Brox. Octree generating networks: Efficient convolutional architectures for high-resolution 3d outputs. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2088–2096, 2017.

[7] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. *Advances in Neural Information Processing Systems*, 30, 2017.

[8] Lijun Yu, Cheng-Yu Liu, Xin Chen, Jie Li, and Jiashi Liu. Language-image models with 2d tokenization and visual autoregression. In *Advances in Neural Information Processing Systems*, 2024.